Java

Encapsulation,inheritance

```
 class Car {

    private int speed =100 ;
    int mileage = 40;
    String name ="BMW";
    Car(){
        System.out.println("Car class has been called");
    }

    void display(){
        System.out.println(speed +" "+mileage +" "+name);
    }
    public void setName(int speed){
        this.speed = speed;
    }
    // getter
    public int getName(){
        return speed;
    }

}


class Vehicle{
    void start(){
        System.out.println("Bike Started");
    }
}
//single inheritance
class Bike extends Vehicle {
    void twoWheel(){
        System.out.println("Two wheel");
    }
}
class Bus extends Vehicle{
    void fourWheel(){
        System.out.println("Four WHeel");
    }
}
class Auto extends Bike{
    void threeWheel(){
        System.out.println("Three wheel");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        //inheritance => parent -> child
        // single inheritance => parent -> child
        // multilevel inheritance => parent ->child1-> child2
        // Hierarchical inheritance => parent -> child, parent ->child2

        Bus volvo = new Bus();
        volvo.fourWheel();
        volvo.start();

        Auto bajaj = new Auto();
        bajaj.threeWheel();
        bajaj.twoWheel();
        bajaj.start();

        Bike access = new Bike();
        access.twoWheel();
        access.start();


        Car obj = new Car();
        // System.out.println(obj.speed);
        // obj.speed = 10;
        obj.setName(30);
        // obj.mileage=10;
        // obj.name ="BENZ";
        obj.display();
        // int result = obj.getName();
        // System.out.println(result);
       System.out.println(obj.getName());


        //encapsulation -> binding data with method , data security
        //public setter and public getter

        // varialbe  => field
        // function => method
    }
}
```

Theory Notes

1. Encapsulation

- **Definition**: Wrapping data (fields) and methods together, restricting direct access to fields.
- Achieved using:
  - private fields → hidden from outside classes.
  - public setters/getters → controlled access.

2. Inheritance

- **Single Inheritance** → one parent, one child. Example: Bike extends Vehicle.
- **Multilevel Inheritance** → chain of inheritance. Example: Auto extends Bike extends Vehicle.
- **Hierarchical Inheritance** → one parent, multiple children. Example: Bus and Bike both extend Vehicle.
- **Benefits**:
  - Code reuse.
  - Logical hierarchy.
  - Polymorphism (same method behaves differently in subclasses).

3. this Keyword

- Refers to the current object.
- Used to differentiate between instance variables and parameters.
- Example:

java

```
public void setName(String name) {

    this.name = name;

}

class Bike extends Vehicle {

    @Override

    void start() { System.out.println("Bike started"); }

}
```

Practice Tasks

Short Answer

1. What is the difference between single and hierarchical inheritance?
2. Why is encapsulation important in OOP?

3. What does the this keyword represent?

Coding Exercises

1. Create a Student class with private fields id, name, and marks. Add constructors, setters/getters, and a display() method.
2. Extend Vehicle into Car and override the start() method.
3. Demonstrate **multilevel inheritance** with Animal → Mammal → Dog.
4. Add a calculateMileage() method in Car that returns mileage based on speed.
5. Create a Truck class extending Vehicle and add a method loadCapacity().
6. Write a program that shows hierarchical inheritance with Teacher and Student both extending Person.
7. Demonstrate encapsulation by restricting direct access to Car.speed and using setters/getters.